Phase 1 - Target Selection & Environment Preparation

1. Target System Description and Setup Evidence

a) <u>Name and Source:</u>

The selected vulnerable system is **MR Robot VM**, obtained from VulnHub. It is an intentionally vulnerable virtual machine designed for penetration testing practice.

b) <u>Reason for Choosing:</u>

The MR Robot VM was selected because:

- It is specifically designed for security learning and ethical hacking practice
- It contains multiple vulnerabilities suitable for beginners and intermediate learners
- It simulates a real-world web application environment, making it useful for practical understanding of attacks

c) <u>Setup Description:</u>

The MR Robot virtual machine was successfully imported into Oracle VirtualBox using the **.ova** file. The system booted correctly and displayed a Linux login terminal, indicating that the target environment is running properly.

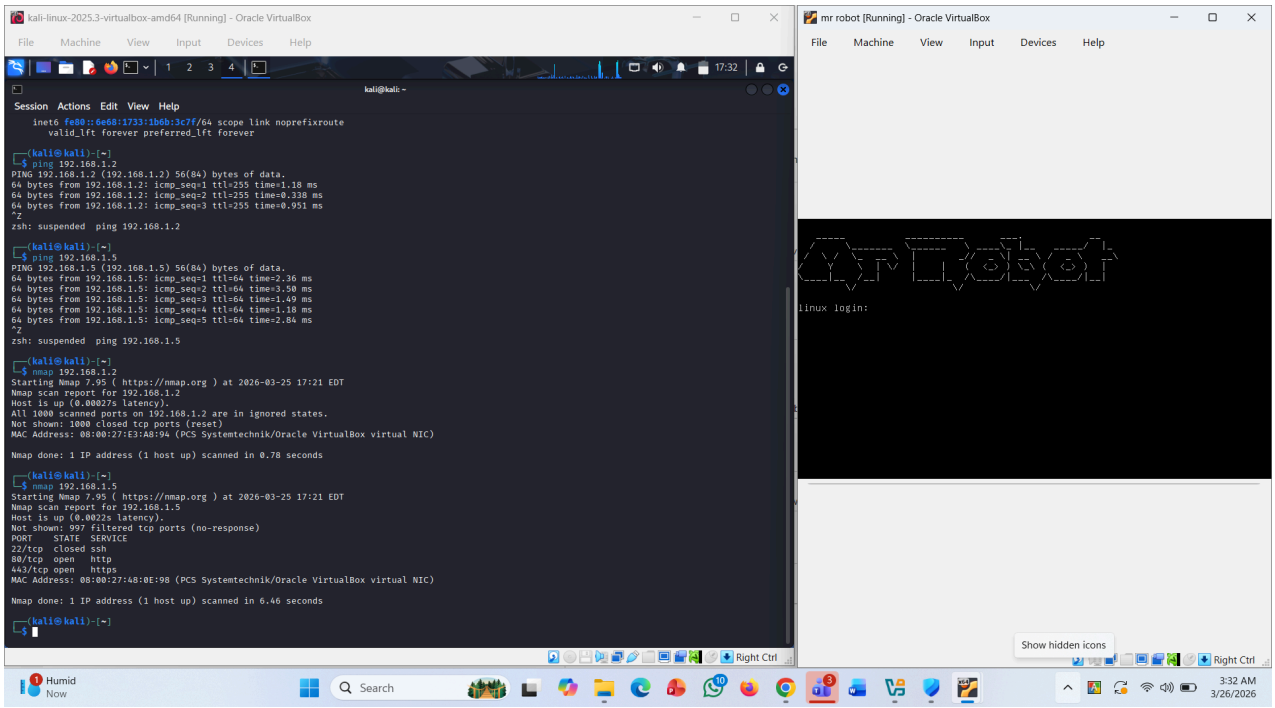

Figure-1: MR Robot & Kali virtual machine successfully running in VirtualBox

2. Network Setup Evidence

a) <u>NAT Network Configuration:</u>

A new NAT Network named **NetSec** was created in VirtualBox to isolate the lab environment from the default network.

- Network Name: NetSec
- Network Range: 192.168.1.0/24
- DHCP: Enabled

This ensures both Kali Linux and the target system can communicate within a controlled environment.

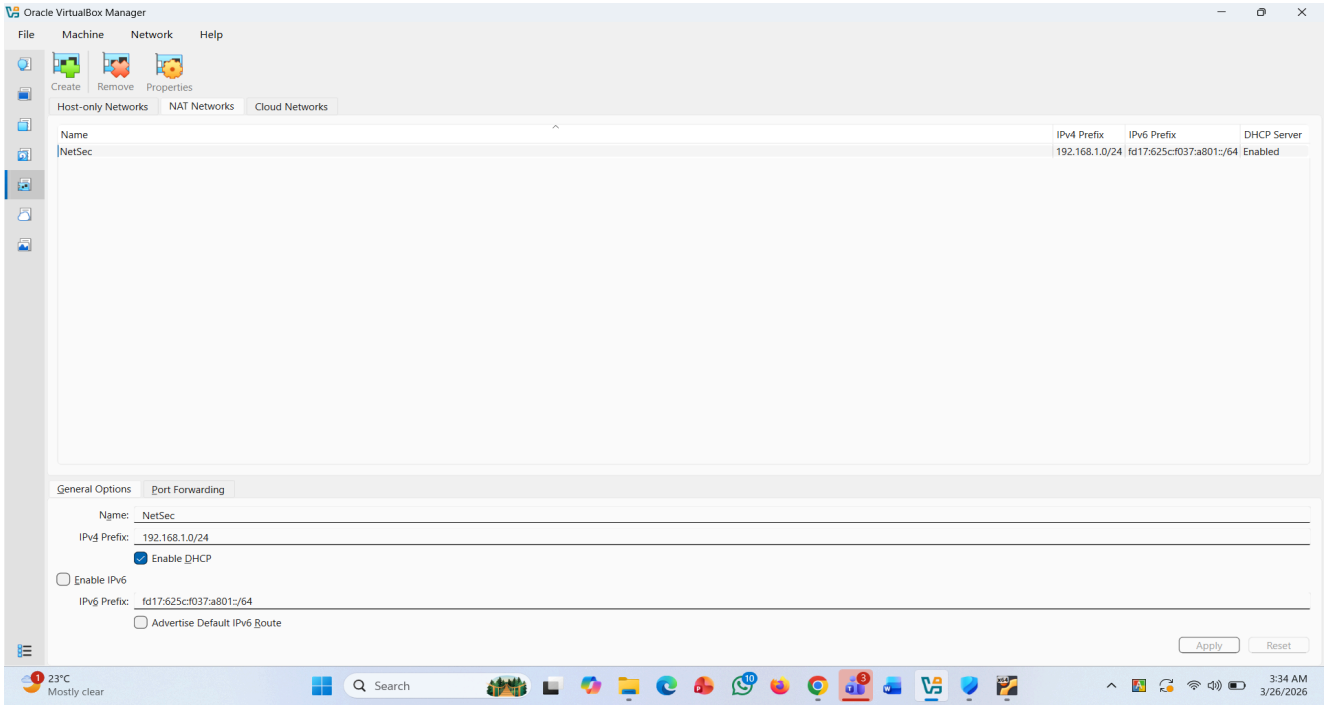

Figure-2: NAT Network configuration in VirtualBox

b) <u>IP Address Assignment and Verification:</u>

Both Kali Linux and MR Robot were connected to the same NAT network. IP addresses were assigned dynamically using DHCP.

- Kali Linux IP: **192.168.1.3**
- MR Robot IP: **192.168.1.5**

The IP addresses were identified using network discovery tools such as netdiscover and verified using the **ping** command.

```
┌──(kali㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:1f:b7:23 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.3/24 brd 192.168.1.255 scope global dynamic noprefixroute eth0
       valid_lft 301sec preferred_lft 301sec
    inet6 fe80::6e68:1733:1b6b:3c7f/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

Figure-3: IP address of Kali Linux

```
Currently scanning: 192.168.0.0/16   |   Screen View: Unique Hosts

3 Captured ARP Req/Rep packets, from 3 hosts.   Total size: 184
_____________________________________________________________________________
  IP            At MAC Address     Count     Len  MAC Vendor / Hostname
_____________________________________________________________________________

192.168.1.1     52:55:c0:a8:01:01      1      64  Unknown vendor
192.168.1.2     08:00:27:e3:a8:94      1      60  PCS Systemtechnik GmbH
192.168.1.5     08:00:27:48:0e:98      1      60  PCS Systemtechnik GmbH
```

Figure-4: Network discovery showing active hosts including MR Robot

c) **Communication Verification:**

Connectivity between Kali Linux and the MR Robot VM was verified using the **ping** command. Successful responses confirmed that both systems are properly connected within the same network.

```
┌──(kali㉿kali)-[~]
└─$ ping 192.168.1.5
PING 192.168.1.5 (192.168.1.5) 56(84) bytes of data.
64 bytes from 192.168.1.5: icmp_seq=1 ttl=64 time=4.24 ms
64 bytes from 192.168.1.5: icmp_seq=2 ttl=64 time=2.87 ms
64 bytes from 192.168.1.5: icmp_seq=3 ttl=64 time=0.992 ms
64 bytes from 192.168.1.5: icmp_seq=4 ttl=64 time=1.21 ms
64 bytes from 192.168.1.5: icmp_seq=5 ttl=64 time=2.72 ms
64 bytes from 192.168.1.5: icmp_seq=6 ttl=64 time=1.11 ms
64 bytes from 192.168.1.5: icmp_seq=7 ttl=64 time=1.83 ms
64 bytes from 192.168.1.5: icmp_seq=8 ttl=64 time=1.12 ms
^Z
zsh: suspended  ping 192.168.1.5
```

Figure-5: Successful communication between Kali Linux and MR Robot

Phase 2 - SYSTEM DISCOVERY

3. System Discovery Results

In this phase, the target system was analyzed from a network perspective without any prior knowledge of its configuration, as required by the assignment. Network scanning and

enumeration techniques were used to identify open ports, running services, and potential security weaknesses.

a) Target Identification:

The target system was identified within the isolated virtual network using the **netdiscover** tool, which performs ARP-based network discovery. The following command was used:

sudo netdiscover

The scan revealed an active host with the IP address: **192.168.1.5**

The MAC address associated with this host corresponds to an Oracle VirtualBox virtual network interface, confirming it as the selected target virtual machine.

```
Currently scanning: 192.168.0.0/16   |   Screen View: Unique Hosts

3 Captured ARP Req/Rep packets, from 3 hosts.   Total size: 184
_____________________________________________________________________________
  IP            At MAC Address     Count     Len  MAC Vendor / Hostname
_____________________________________________________________________________
192.168.1.1     52:55:c0:a8:01:01      1      64  Unknown vendor
192.168.1.2     08:00:27:e3:a8:94      1      60  PCS Systemtechnik GmbH
192.168.1.5     08:00:27:48:0e:98      1      60  PCS Systemtechnik GmbH
```

Figure-6: Finding the target ip

b) Port Scanning:

A comprehensive scan was conducted using Nmap to determine open ports and accessible services on the target system. The following command was used:

nmap -sC -sV 192.168.1.5

The results showed:

- Port 80/tcp — Open
- Port 443/tcp — Open
- Port 22/tcp — Closed

These results indicate that the system primarily exposes web-based services over HTTP and HTTPS, while remote access through SSH is not available.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sC -sV 192.168.1.5
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-25 17:47 EDT
Nmap scan report for 192.168.1.5
Host is up (0.0014s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT    STATE  SERVICE  VERSION
22/tcp  closed ssh
80/tcp  open   http     Apache httpd
|_http-title: Site doesn't have a title (text/html).
|_http-server-header: Apache
443/tcp open   ssl/http Apache httpd
|_http-server-header: Apache
|_http-title: Site doesn't have a title (text/html).
| ssl-cert: Subject: commonName=www.example.com
| Not valid before: 2015-09-16T10:45:03
|_Not valid after:  2025-09-13T10:45:03
MAC Address: 08:00:27:48:0E:98 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 25.50 seconds
```

Figure-7: Port scanning with Nmap

c) <u>Service Identification:</u>

Further service detection and technology fingerprinting were performed using **WhatWeb**. The following command was used:

whatweb 192.168.1.5

The analysis revealed:

- The system is running an Apache HTTP server
- HTTPS service is also provided over Apache
- The Apache mod_pagespeed module is enabled (identified via **x-mod-pagespeed** header)

These findings confirm that the primary attack surface of the target system is its web application.

```
┌──(kali㉿kali)-[~]
└─$ whatweb 192.168.1.5
http://192.168.1.5 [200 OK] Apache, Country[RESERVED][ZZ], HTML5, HTTPServer[Apache], IP[192.168.1.5], Script, UncommonHeaders[x-mod-pagespeed],
X-Frame-Options[SAMEORIGIN]
https://192.168.1.5 [200 OK] Apache, Country[RESERVED][ZZ], HTML5, HTTPServer[Apache], IP[192.168.1.5], Script, UncommonHeaders[x-mod-pagespeed],
 X-Frame-Options[SAMEORIGIN]
```

Figure-8: Technology fingerprinting using Whatweb

d) <u>Identification of Potential Weaknesses:</u>

Further enumeration was conducted using **Nikto** to identify security issues in the web server configuration. The following command was used:

nikto -h 192.168.1.5

The scan revealed multiple potential weaknesses:

- Presence of WordPress login interfaces such as **/wp-login.php** and **/wp-admin/**
- Exposure of sensitive configuration file reference: **#wp-config.php#**
- Apache MultiViews enabled (mod_negotiation)
- Missing security headers such as **X-Content-Type-Options**
- WordPress cookie created without HttpOnly flag
- Additional administrative paths such as **/admin/** and **/readme**

These findings indicate that the system may be vulnerable to credential-based attacks, information disclosure, and misconfiguration-based exploitation.

```
┌──(kali㉿kali)-[~]
└─$ nikto -h 192.168.1.5
- Nikto v2.5.0
---------------------------------------------------------------------------
+ Target IP:          192.168.1.5
+ Target Hostname:    192.168.1.5
+ Target Port:        80
+ Start Time:         2026-03-25 17:52:07 (GMT-4)
---------------------------------------------------------------------------
+ Server: Apache
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to th
e MIME type. See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ /zrPWVCcA.cwr: Retrieved x-powered-by header: PHP/5.5.29.
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ /index: Uncommon header 'tcn' found, with contents: list.
+ /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The following alternatives
for 'index' were found: index.html, index.php. See: http://www.wisec.it/sectou.php?id=4698ebdc59d15,https://exchange.xforce.ibmcloud.com/vulnerab
ilities/8275
+ /admin/: This might be interesting.
+ /readme: This might be interesting.
+ /image/: Drupal Link header found with value: <http://192.168.1.5/?p=23>; rel=shortlink. See: https://www.drupal.org/
+ /wp-links-opml.php: This WordPress script reveals the installed version.
+ /license.txt: License file found may identify site software.
+ /admin/index.html: Admin login page/section found.
+ /wp-login/: Cookie wordpress_test_cookie created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ /wp-login/: Admin login page/section found.
+ /wordpress/: A Wordpress installation was found.
+ /wp-admin/wp-login.php: Wordpress login found.
+ /wordpress/wp-admin/wp-login.php: Wordpress login found.
+ /blog/wp-login.php: Wordpress login found.
+ /wp-login.php: Wordpress login found.
+ /wordpress/wp-login.php: Wordpress login found.
+ /#wp-config.php#: #wp-config.php# file found. This file contains the credentials.
+ 8102 requests: 0 error(s) and 19 item(s) reported on remote host
+ End Time:           2026-03-25 17:56:29 (GMT-4) (262 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

Figure-9: Enumeration using nikto

e) **<u>Identification of Outdated or Insecure Services:</u>**

Based on the results obtained from WhatWeb and Nikto, the following services and configurations appear insecure or potentially outdated:

- WordPress Application Exposure (High Risk)

Nikto detected multiple WordPress-related endpoints including **/wordpress/, /wp-login.php,** and **/wp-admin/**. WordPress systems are commonly targeted due to weak authentication mechanisms and known vulnerabilities.

- Exposure of Sensitive Configuration File (Critical Risk)

Nikto identified the presence of: **#wp-config.php#**

This file may contain database credentials and internal configuration details, posing a serious security risk if accessible.

- Outdated PHP Version

Nikto detected the following header: **PHP/5.5.29**

This version is outdated and unsupported, making it vulnerable to known exploits.

- Apache Web Server Configuration Issues

Apache was detected running on both HTTP and HTTPS ports. The server configuration shows weaknesses such as missing security headers and exposed administrative directories.

- Apache MultiViews Enabled (Information Disclosure Risk)

Nikto reported Apache mod_negotiation (MultiViews) enabled, which may allow attackers to discover hidden files through brute-force techniques.

- mod_pagespeed Module Detected (Potential Risk)

WhatWeb identified the Apache mod_pagespeed module through the **x-mod-pagespeed** header. While useful for performance optimization, outdated or misconfigured versions may introduce additional attack surface. No direct exploitation was observed.

- Missing Security Header (X-Content-Type-Options)

Nikto reported the absence of this header, which may allow MIME-type sniffing attacks.

- WordPress Cookie Without HttpOnly Flag

Nikto identified that a WordPress test cookie was created without the HttpOnly flag, increasing the risk of session theft if other vulnerabilities are present.

f) <u>Summary of Findings:</u>

The system discovery process revealed that the target system exposes web services over HTTP and HTTPS. The application appears to be based on WordPress running on Apache and contains multiple configuration weaknesses and outdated components. These findings indicate a high likelihood of successful exploitation through web-based attack techniques.

Phase 3 - Observation of Network Communication

4. Network Communication Observation

a) <u>Traffic Generation:</u>

Network activity was generated by interacting with the WordPress login page of the target system. The login interface was accessed through:

http://192.168.1.5/wp-login.php

Test credentials were entered to produce HTTP POST traffic between the attacker system (Kali Linux) and the target system.

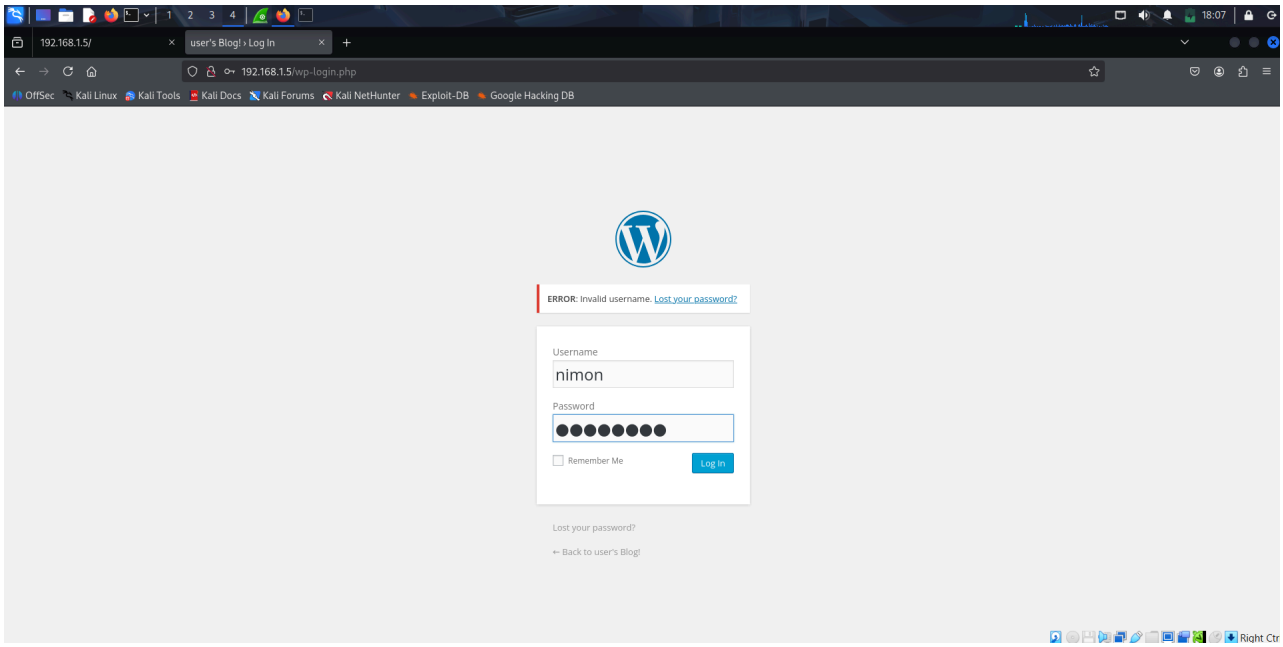

Figure-10: WordPress login interface of the target system used to generate network traffic during the experiment.

b) <u>Captured Traffic Evidence:</u>

Wireshark captured traffic using IP and HTTP filters, showing HTTP GET/POST requests along with underlying TCP communication such as connection establishment and data transfer.

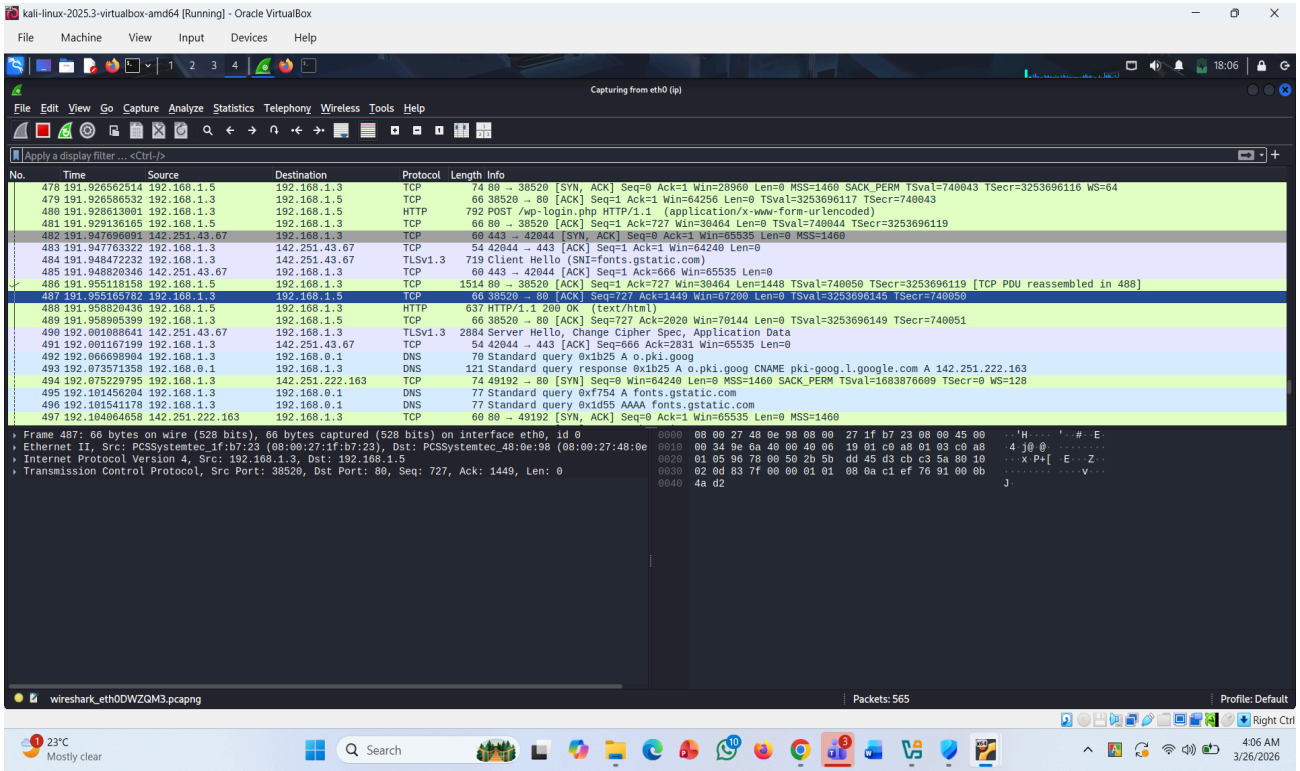

Figure-11: Wireshark capture showing HTTP & TCP traffic generated between the attacker system and the target system after interacting with the web application.

c) <u>Explanation of Visible Information:</u>

Further analysis of captured packets revealed that the communication was conducted using the **HTTP protocol**, which transmits data in plain text.

Inspection of the HTTP POST request (**/wp-login.php**) showed that login credentials were visible during transmission:

- Username: **nimon**
- Password: **nimon123**

This confirms that sensitive information can be intercepted by an attacker monitoring the network.

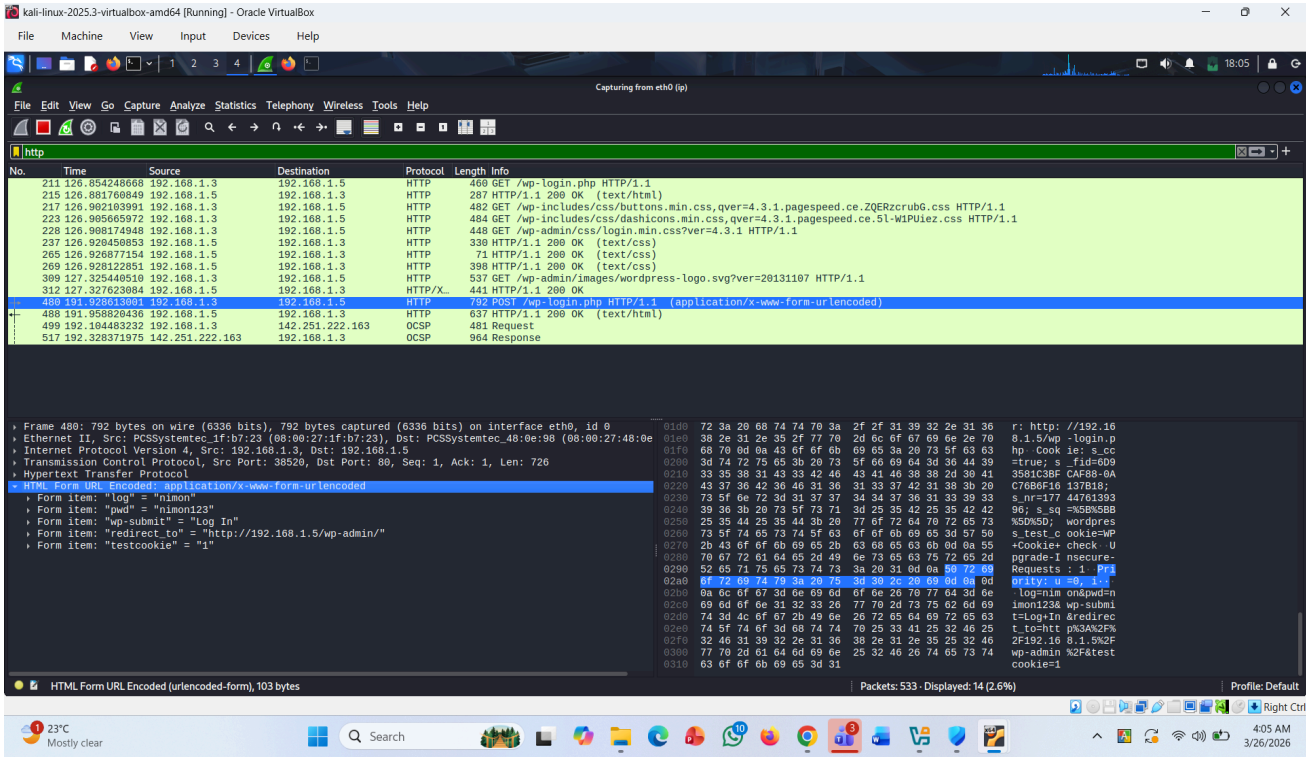

Figure-12: Captured HTTP POST request displaying login credentials transmitted in plain text (username and password).

Phase 4 - Authentication Strength Evaluation

5. Authentication Evaluation

a) <u>Extraction of Authentication Data:</u>

During web enumeration, the **robots.txt** file was analyzed using a request (e.g., via **curl http://192.168.1.5/robots.txt**) and was found to expose sensitive resources. Among these was a wordlist file named **fsocity.dic**. This file was downloaded using the following command:

wget http://192.168.1.5/fsocity.dic

The wordlist contained a large number of potential usernames and passwords, indicating poor security practices and improper exposure of authentication-related data.

```
┌──(kali㉿kali)-[~]
└─$ wget http://192.168.1.5/fsocity.dic

--2026-03-25 18:08:47--  http://192.168.1.5/fsocity.dic
Connecting to 192.168.1.5:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 7245381 (6.9M) [text/x-c]
Saving to: 'fsocity.dic'

fsocity.dic                         100%[===================================================================>]  6.91M  9.27MB/s    in 0.7s

2026-03-25 18:08:48 (9.27 MB/s) - 'fsocity.dic' saved [7245381/7245381]
```

Figure-13: Downloading fsocity.dic using wget

b) <u>Username Verification:</u>

The username **"elliot"** was identified from the wordlist and tested on the WordPress login page. The system response confirmed that the username exists, as it indicated an incorrect password instead of an invalid username.

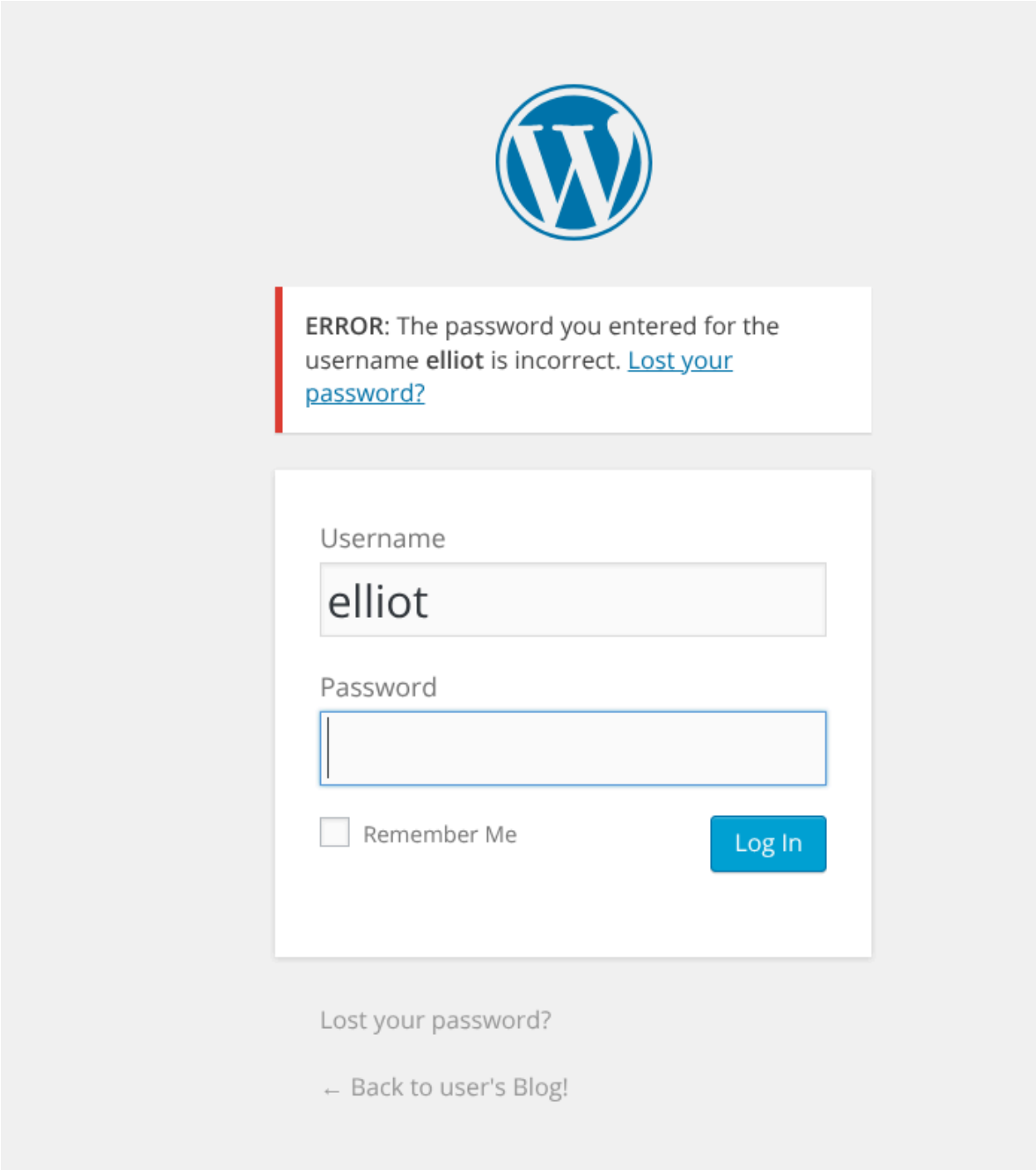

Figure-14: WordPress login response confirming the existence of the username "elliot" through an incorrect password message.

c) <u>Password Recovery:</u>

A dictionary-based password attack was performed using the WPScan tool with the downloaded wordlist. The following command was used:

wpscan --url http://192.168.1.5 -U elliot -P fsocity.dic

The attack successfully recovered valid credentials:

```
┌──(kali㉿kali)-[~]
└─$ wpscan --url http://192.168.1.5 -U elliot -P fsocity.dic
_______________________________________________________________
         __          _______   _____
         \ \        / /  __ \ / ____|
          \ \  /\  / /| |__) | (___   ___  __ _ _ __   ®
           \ \/  \/ / |  ___/ \___ \ / __|/ _` | '_ \
            \  /\  /  | |     ____) | (__| (_| | | | |
             \/  \/   |_|    |_____/ \___|\__,_|_| |_|

         WordPress Security Scanner by the WPScan Team
                         Version 3.8.28

       @_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
_______________________________________________________________

[i] Updating the Database ...
[i] Update completed.

[+] URL: http://192.168.1.5/ [192.168.1.5]
[+] Started: Wed Mar 25 18:14:14 2026

Interesting Finding(s):

[+] Headers
 | Interesting Entries:
 |  - Server: Apache
 |  - X-Mod-Pagespeed: 1.9.32.3-4523
 | Found By: Headers (Passive Detection)
 | Confidence: 100%

[+] robots.txt found: http://192.168.1.5/robots.txt
 | Found By: Robots Txt (Aggressive Detection)
 | Confidence: 100%

[+] XML-RPC seems to be enabled: http://192.168.1.5/xmlrpc.php
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 100%
 | References:
 |  - http://codex.wordpress.org/XML-RPC_Pingback_API
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_ghost_scanner/
 |  - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/

[+] The external WP-Cron seems to be enabled: http://192.168.1.5/wp-cron.php
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 60%
```

Figure-15: Password Recovery Using WPScan

- Username: **elliot**
- Password: **ER28-0652**

```
[!] Valid Combinations Found:
 | Username: elliot, Password: ER28-0652

[!] No WPScan API Token given, as a result vulnerability data has not been output.
[!] You can get a free API token with 25 daily requests by registering at https://wpscan.com/register

[+] Finished: Wed Mar 25 18:45:52 2026
[+] Requests Done: 1908
[+] Cached Requests: 6
[+] Data Sent: 227.076 MB
[+] Data Received: 199.14 MB
[+] Memory used: 242.328 MB
[+] Elapsed time: 00:31:38
```

Figure-16: Password cracked successfully.

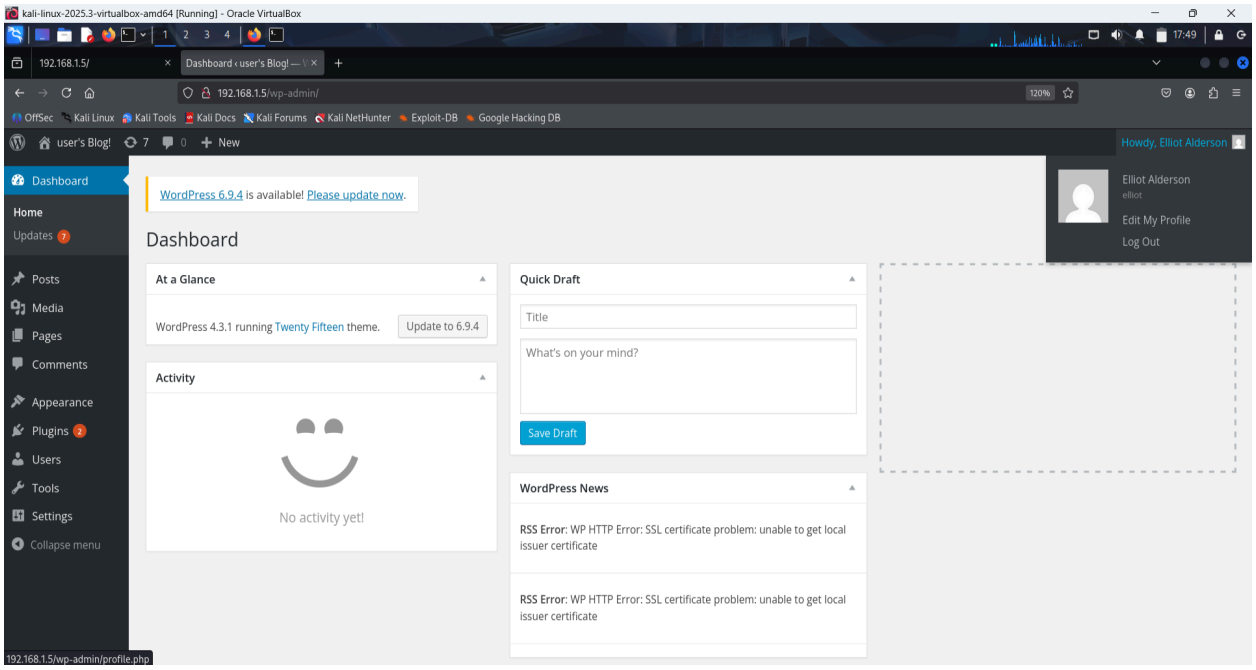

Figure-17: Successful authentication using cracked credentials

d) <u>Evaluation of Authentication Strength:</u>

The successful recovery of credentials demonstrates that the system is vulnerable to dictionary-based password attacks due to weak password selection. The exposure of a large wordlist (fsocity.dic) through the robots.txt file, combined with the ability to enumerate valid usernames, significantly reduces the effort required for an attacker to gain unauthorized access.

This indicates poor authentication design and lack of protective mechanisms such as account lockout or rate limiting.

e) **Conclusion:**

This phase highlights that poor password practices significantly weaken system security. Attackers can recover credentials using publicly exposed wordlists, leading to unauthorized access even without direct access to password hashes.

Phase 5 - Security Analysis & Recommendations

6. Security Recommendations

a) **Identified Security Weaknesses:**

→ **Exposure of Sensitive Resources (robots.txt):** The **robots.txt** file exposed sensitive resources such as the **fsocity.dic** wordlist, which was later used to perform a successful password attack. This weakness occurs due to improper access control and placing sensitive files in publicly accessible locations, which can allow unauthorized users to access confidential information.

→ **Weak Password Authentication:** The password for the user **elliot** was successfully recovered using a dictionary attack (**Password: ER28-0652**). This weakness exists due to the lack of a strong password policy, allowing predictable or weak passwords. Additionally, there is no protection against repeated login attempts, making brute-force or dictionary attacks possible.

→ **Unencrypted Communication (HTTP Usage):** From Phase 3, it was observed that login credentials were transmitted using **HTTP**, allowing sensitive information to be captured in plain text. Evidence showed that the username and password were visible in the **HTTP POST request**. This weakness exists due to the absence of HTTPS implementation and lack of encryption for data in transit, making the credentials vulnerable to interception.

→ **User Enumeration Vulnerability:** The WordPress login page revealed valid usernames through different error messages, such as "**invalid username**" and "**incorrect password**." This weakness exists due to improper authentication response handling. Additionally, the lack of generic error messages allows attackers to identify valid usernames.

→ **Outdated and Misconfigured Services:** From Phase 2 analysis, the system contained multiple insecure configurations:

- Outdated PHP version (PHP 5.5.29)
- Apache misconfigurations (MultiViews enabled)
- Missing security headers
- WordPress cookie without HttpOnly flag
- Exposure of admin paths and configuration references

This weakness exists due to a lack of regular system updates and poor server configuration and hardening.

b) <u>Explanation of Weaknesses:</u>

The identified vulnerabilities exist due to weaknesses across multiple layers of the system:

- **Application Layer:** Weak authentication and exposed files
- **Network Layer:** Lack of encryption (HTTP communication)
- **System Layer:** Outdated software and insecure configurations

These issues collectively enabled a complete attack chain:

Network Discovery → Service Enumeration → File Exposure → Username Identification → Password Attack → Credential Recovery

This demonstrates how multiple small weaknesses can combine to create a serious security risk.

c) <u>Recommended Countermeasures:</u>

→ Stronger Authentication:

- Enforce strong password policies (minimum length and complexity)
- Implement account lockout after multiple failed login attempts
- Enable Multi-Factor Authentication (MFA)

→ Use of Encrypted Communication:

- Replace HTTP with HTTPS
- Implement valid SSL/TLS certificates
- Ensure all authentication data is transmitted securely

→ System Updates:

- Upgrade outdated PHP version
- Update Apache server and WordPress installation
- Apply regular security patches

→ **Access Control and Service Hardening:**

- Restrict access to sensitive files (robots.txt, configuration files)
- Remove or secure exposed resources such as wordlists
- Disable unnecessary services and modules (e.g., MultiViews)
- Apply proper file permissions

→ **Secure Configuration Practices:**

- Implement security headers (e.g., X-Content-Type-Options)
- Set cookies with HttpOnly and Secure flags
- Avoid exposing internal system details

→ **Prevent User Enumeration:**

- Use generic login error messages
- Do not reveal whether a username exists

7. Task Distribution Among Group Members

Name	ID	Contribution	Percentage
DIPRA KAR	22-46336-1	SYSTEM DISCOVERY	20%
MONJURUL ISLAM NIMON	23-50004-1	Observation of Network Communication, Authentication Strength Evaluation(password cracking) and overall documentation	40%
PRITHIRAJ MODAK	23-50007-1	Target Selection, Environment Preparation and overall documentation	20%
MD :SADMAN SAKIB	23-50039-1	Security Analysis & Recommendations	20%